

Module 15: ALTER TABLE Statement

The **ALTER TABLE** statement is used to modify the structure of an existing table. It allows you to add, remove, or modify columns and constraints without recreating the table.



Introduction

In real-world databases, requirements change frequently.

Examples:

- Company wants to store employee emails.
- Product table needs a new category column.
- Customer name length must increase.
- New business rules require constraints.

Instead of deleting and recreating the table, SQL provides:

ALTER TABLE



Learning Objectives

After completing this module, you will be able to:

- ✓ Add new columns
 - ✓ Modify existing columns
 - ✓ Drop columns
 - ✓ Add constraints
 - ✓ Remove constraints
 - ✓ Understand schema evolution
 - ✓ Apply ALTER TABLE safely in production
-



Table of Contents

1. What is ALTER TABLE?
2. Why ALTER TABLE is Important

3. ALTER TABLE Syntax
4. ADD COLUMN
5. ALTER COLUMN
6. DROP COLUMN
7. ADD CONSTRAINT
8. DROP CONSTRAINT
9. Working with Existing Data
10. Common Errors
11. Best Practices
12. Real-World Examples
13. Summary
14. Practice Questions

1 What is ALTER TABLE?

ALTER TABLE modifies an existing table structure.

Example

Original Table

```
CREATE TABLE Employees
(
    EmployeeID INT,
    EmployeeName VARCHAR(100)
);
```

Need:

```
Employee Email
```

Solution:

```
ALTER TABLE Employees
ADD Email VARCHAR(100);
```

Updated Structure

Column

EmployeeID

EmployeeName

Email

Why ALTER TABLE Matters

Used for:

- New business requirements
- Database upgrades
- Data model improvements
- Production maintenance

2 Why ALTER TABLE is Important

Imagine:

Initial Design

EmployeeID
EmployeeName

After 6 months:

Need:

Email
JoiningDate
Department

Without ALTER TABLE:

Recreate entire table
Move all data
Risk data loss

With ALTER TABLE:

```
ALTER TABLE Employees  
ADD Email VARCHAR(100);
```

Quick and safe.

3 ALTER TABLE Syntax

General syntax:

```
ALTER TABLE TableName  
Action;
```

Actions include:

```
ADD  
ALTER COLUMN  
DROP COLUMN  
ADD CONSTRAINT  
DROP CONSTRAINT
```

4 ADD COLUMN

Used to add new columns.

Syntax

```
ALTER TABLE TableName  
ADD ColumnName DataType;
```

Example

```
ALTER TABLE Employees
```

```
ADD Email VARCHAR(100);
```

Before

EmployeeID	EmployeeName
------------	--------------

After

EmployeeID	EmployeeName	Email
------------	--------------	-------

Add Multiple Columns

```
ALTER TABLE Employees
ADD
    City VARCHAR(50),
    Country VARCHAR(50);
```

Result

New columns added simultaneously.

Add Column with Default

```
ALTER TABLE Employees
ADD Country VARCHAR(50)
DEFAULT 'India';
```

5 ALTER COLUMN

Used to modify an existing column.

Syntax

```
ALTER TABLE TableName
ALTER COLUMN ColumnName NewDataType;
```

Example

Current:

```
EmployeeName VARCHAR(50)
```

Need:

```
EmployeeName VARCHAR(100)
```

Query

```
ALTER TABLE Employees  
ALTER COLUMN EmployeeName VARCHAR(100);
```

Common Uses

Increase Length

```
VARCHAR(50)  
→  
VARCHAR(200)
```

Change Data Type

```
INT  
→  
BIGINT
```

Change NULL Property

```
ALTER TABLE Employees  
ALTER COLUMN Email VARCHAR(100) NOT NULL;
```

Important

Existing data must be compatible.

Example

Bad:

```
VARCHAR  
→  
INT
```

when values contain text.

Error occurs.

DROP COLUMN

Removes a column permanently.

Syntax

```
ALTER TABLE TableName  
DROP COLUMN ColumnName;
```

Example

```
ALTER TABLE Employees  
DROP COLUMN Email;
```

Before

EmployeeID	EmployeeName	Email
------------	--------------	-------

After

EmployeeID	EmployeeName
------------	--------------

⚠ Data in Email column is permanently lost.

Drop Multiple Columns

```
ALTER TABLE Employees
DROP COLUMN City,
Country;
```

7 ADD CONSTRAINT

Constraints can be added later.

Add PRIMARY KEY

```
ALTER TABLE Employees
ADD CONSTRAINT PK_Employees
PRIMARY KEY (EmployeeID);
```

Add UNIQUE

```
ALTER TABLE Employees
ADD CONSTRAINT UQ_Email
UNIQUE (Email);
```

Add CHECK

```
ALTER TABLE Employees
ADD CONSTRAINT CHK_Salary
CHECK (Salary > 0);
```

Add DEFAULT

```
ALTER TABLE Employees
ADD CONSTRAINT DF_Country
DEFAULT 'India'
FOR Country;
```

Add FOREIGN KEY

```
ALTER TABLE Employees
ADD CONSTRAINT FK_Department
FOREIGN KEY (DepartmentID)
REFERENCES Departments(DepartmentID);
```

DROP CONSTRAINT

Remove existing constraints.

Syntax

```
ALTER TABLE TableName
DROP CONSTRAINT ConstraintName;
```

Example

```
ALTER TABLE Employees
DROP CONSTRAINT UQ_Email;
```

Constraint removed.

Find Constraint Names

SQL Server:

```
sp_help Employees;
```

or

```
EXEC sp_help 'Employees';
```

9 Working with Existing Data

Most important production concept.

Example

Table:

EmployeeID	Salary
101	50000
102	60000

Adding:

```
CHECK (Salary > 0)
```

works because all existing rows satisfy rule.

Example Failure

Existing Data:

Salary
-1000

Adding:

```
CHECK (Salary > 0)
```

fails.

Reason:

Existing records violate new constraint.

Production Rule

Always verify existing data before schema changes.

10 Common Errors

Data Type Conversion Failure

Bad:

```
ALTER COLUMN Salary INT;
```

when values contain:

```
45000.75
```

NOT NULL Failure

Bad:

```
ALTER COLUMN Email VARCHAR(100) NOT NULL;
```

when existing NULL values exist.

Dropping Used Columns

Bad:

```
DROP COLUMN DepartmentID;
```

when Foreign Keys depend on it.

Adding Duplicate Constraint

Constraint names must be unique.

Best Practices

Backup Before Changes

Especially in production.

Test in Development

Never alter production first.

Check Existing Data

Before:

```
NOT NULL  
CHECK  
FOREIGN KEY
```

changes.

Use Meaningful Constraint Names

Good:

```
PK_Employees  
FK_Employees_Departments
```

```
CHK_Salary
```

Avoid Unnecessary Schema Changes

Frequent changes increase risk.

Document Every Change

Professional databases track schema evolution.

Professional Workflow

Step 1

Review structure.

```
sp_help Employees;
```

Step 2

Check data.

```
SELECT *  
FROM Employees;
```

Step 3

Apply change.

```
ALTER TABLE Employees  
ADD Email VARCHAR(100);
```

Step 4

Verify.

```
sp_help Employees;
```

1 2 Real-World Examples

Add Email Column

```
ALTER TABLE Employees
ADD Email VARCHAR(100);
```

Increase Product Name Length

```
ALTER TABLE Products
ALTER COLUMN ProductName VARCHAR(200);
```

Add Salary Validation

```
ALTER TABLE Employees
ADD CONSTRAINT CHK_Salary
CHECK (Salary > 0);
```

Remove Temporary Column

```
ALTER TABLE Employees
DROP COLUMN TempData;
```

Add Department Relationship

```
ALTER TABLE Employees
ADD CONSTRAINT FK_Department
```

```
FOREIGN KEY (DepartmentID)
REFERENCES Departments(DepartmentID);
```

Complete Example

```
ALTER TABLE Employees
ADD Email VARCHAR(100);

ALTER TABLE Employees
ADD CONSTRAINT UQ_Email
UNIQUE (Email);
```



Summary

In this module, you learned:

- ✓ ALTER TABLE
- ✓ ADD COLUMN
- ✓ ALTER COLUMN
- ✓ DROP COLUMN
- ✓ ADD CONSTRAINT
- ✓ DROP CONSTRAINT
- ✓ Schema Evolution
- ✓ Production Safety Practices
- ✓ Common ALTER TABLE Errors



Practice Questions

Theory

1. What is ALTER TABLE?
2. Why use ALTER TABLE?
3. How do you add a column?
4. How do you modify a column?
5. How do you remove a column?

6. How do you add a constraint?
 7. How do you drop a constraint?
 8. What happens when dropping a column?
 9. Why check existing data first?
 10. What are ALTER TABLE best practices?
-

Practical Exercises

Task 1

Add:

Email

column to Employees.

Task 2

Increase EmployeeName length from:

`VARCHAR(50)`

to

`VARCHAR(100)`

Task 3

Add:

`UNIQUE`

constraint to Email.

Task 4

Add:


```
CHECK (Salary > 0)
```

constraint.

Task 5

Drop a column.

Task 6

Drop a constraint.

Challenge Project

Create:

```
Employees  
Departments
```

Then:

- Add new columns
- Modify data types
- Add primary keys
- Add foreign keys
- Add check constraints
- Remove unused columns

using only ALTER TABLE statements.



Next Module

→ Module 16: SQL JOINS (Most Important SQL Topic)

Topics Covered:

- Why JOINS Are Needed
- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- SELF JOIN
- CROSS JOIN

- Join Execution Logic
- Real-World Analytics Queries
- Interview Questions